
Clase 191 — Synthetic Control Method dedicado (pysyncon, SparseSC)

Parte: 3 — Estadística Inferencial y Causal · Fuente: Abadie, Diamond & Hainmueller (2010) + Doudchenko & Imbens (2016) + Arkhangelsky et al. (2021) Synthetic DiD. Duración estimada: 80 min. · Nivel: Avanzado

Clase 191 — Synthetic Control Method dedicado (pysyncon, SparseSC)

Parte: 3 — Estadística Inferencial y Causal · Fuente: Abadie, Diamond & Hainmueller (2010) + Doudchenko & Imbens (2016) + Arkhangelsky et al. (2021) Synthetic DiD. Duración estimada: 80 min. · Nivel: Avanzado

Objetivo

Aplicar Synthetic Control Method (Abadie et al. 2010) — el estándar para evaluar políticas o intervenciones aplicadas a una única unidad (un país, un estado, una ciudad) sin grupo control natural. Construir un "control sintético" como combinación ponderada de donors. Conocer variantes modernas: Synthetic DiD (Arkhangelsky 2021), Generalized SC (Xu 2017), SparseSC (Microsoft Research).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Construir un Synthetic Control con pysyncon: tratado, donors, predictors, períodos pre/post.
- Interpretar pesos W (combinación convexa) y path plot.
- Aplicar placebo test (in-time y in-space) como inference informal.
- Conocer Synthetic DiD que combina lo mejor de DiD y SCM.
- Reconocer cuándo SCM no aplica (pocos donors, fit pre malo).

Temas

- Setup: 1 tratado + N donors + features predictoras + período pre/post.
- Optimización: pesos minimizan $\|Y_{\text{treat_pre}} - W \cdot Y_{\text{donors_pre}}\|^2$.
- Constraint: $w_i \geq 0$, $\sum w_i = 1$ (combinación convexa) — clásico.
- Placebo test in-space: aplicar SCM a cada donor; comparar effect real vs distribución de placebos.
- Placebo in-time: aplicar antes del tratamiento real → debería dar 0.
- Synthetic DiD: relax constraints + agregar pesos temporales.

Definiciones y características

- Tratado: la única unidad que recibió intervención.
- Donors: pool de unidades no tratadas, idealmente similares pre.
- Predictores: covariables usadas para hacer matching pre.
- Dataprep: clase pysyncon que estructura input.
- Synth: optimizador que encuentra pesos.
- Pre-RMSPE: error de matching pre-período. Si alto, malo.
- Post-RMSPE / Pre-RMSPE ratio: indicador informal de magnitud del efecto.

Dataset / recursos

- California Prop 99 smoking (Abadie's dataset clásico).
- Cualquier panel de países $\times$ años con una intervención.
- Librerías: pysyncon, SparseSC (Microsoft), numpy, pandas.

Ejercicios

1. California Prop 99: cargar dataset, definir tratado California, donors otros estados, pre 1970-1988, post 1989-2000.
2. Path plot: `synth.path_plot()` — California real vs sintética. Visualizar gap post-1989.
3. Pesos: imprimir `synth.weights`. Verificar que solo few estados tienen peso > 0 .
4. Placebo in-space: aplicar a cada otro estado; plot de gaps. California debe destacar.
5. Placebo in-time: tratamiento artificial en 1980 (5 años antes del real). Gap debería ser ≈ 0 .

Homework verifiable

Estudio de caso: efecto de una política aplicada a 1 unidad (ej.: Brexit en UK, COVID lockdowns en una ciudad):

1. Dataset panel, 10+ donors, ≥ 5 años pre.
2. SCM con pysyncon.
3. Path + gap plots.
4. Placebo in-space.
5. Interpretación: ¿el efecto observado está en el extremo de la distribución placebo?

Criterio de aceptación: pre-RMSPE bajo (good fit); diagnóstico de placebo informativo; conclusión justificada.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Pre-RMSPE alto	Donors no comparables. Fix: filtrar donors
Pocos donors (<10)	Pesos no robustos. Fix: ampliar donor pool
Predictores correlated $\rightarrow$ pesos inestables	Fix: SparseSC con regularización.
Reportar p-value clásico	No existe en SCM. Fix: placebo test ranks
Aplicar SCM a tratamiento gradual	SCM asume tratamiento discreto. Fix: Synth

Preguntas frecuentes

SCM vs DiD?

DiD necesita "parallel trends"; SCM construye control sintético. SCM mejor cuando 1 tratado; DiD mejor con many.

Synthetic DiD cuándo?

Combina virtudes: usable con many tratados + permite no-parallel trends. Default 2024+ en muchos casos.

Cuántos períodos pre necesarios?

5-10 mínimo. Más es mejor para fit confiable.

SparseSC?

Microsoft Research lib que extiende SCM a paneles grandes con regularización L2.

Bayesian SCM?

Existe (Bayesian Structural Time Series — CausalImpact de Google). Otra alternativa.

Referencias

- Abadie, Diamond & Hainmueller (2010), Synthetic Control Methods for Comparative Case Studies, JASA.
- Doudchenko & Imbens (2016), Balancing, Regression, Difference-in-Differences and Synthetic Control Methods.
- Arkhangelsky et al. (2021), Synthetic Difference-in-Differences, AER.
- pysyncon.
- SparseSC (Microsoft).
- Google CausalImpact (R / Python ports).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Siguiente clase

Clase 192 — Bayes intro: priors, posterior, MCMC con PyMC

Apéndice: notebook (primer bloque)

Abadie et al.: para una unidad tratada (ej: California aprobó Prop99, ¿efecto en consumo de tabaco?), construir un control sintético como combinación convexa de unidades no tratadas que matchea el pre-tratamiento. El gap post = efecto causal. Requiere: pip install numpy scipy matplotlib (opcional pysyncon).

```
import numpy as np
from scipy.optimize import minimize
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
N_control = 20
T_pre, T_post = 10, 10
T = T_pre + T_post

# Generamos N_control trayectorias con tendencia común + ruido
trend = np.linspace(50, 60, T)
controls = trend[None, :] + rng.normal(0, 1, (N_control, 1)) np.arange(T)[None, :] 0.1 \
    + rng.normal(0, 2, (N_control, T))
# Unidad tratada: combinación de 3 controles + ruido en pre, + efecto en post
true_weights = np.zeros(N_control); true_weights[[2, 7, 13]] = [0.4, 0.35, 0.25]
treated_pre = controls[:, :T_pre].T @ true_weights + rng.normal(0, 0.5, T_pre)
treated_post = controls[:, T_pre:].T @ true_weights + rng.normal(0, 0.5, T_post)
```

```
TRUE_EFFECT = 5.0
treated_post += TRUE_EFFECT # efecto aditivo desde t=T_pre
treated = np.concatenate([treated_pre, treated_post])
print(f'Treated shape: {treated.shape} | Controls: {controls.shape} | TRUE effect post = {TRUE_EFFECT}')
```

Archivos complementarios

- notebook.ipynb